

Maxime Albouy
Billal Mouhtadi
Eliot Desplanques

Cahier des charges et Rapports techniques Drone Tracker



Table des matières

I.		3
II.	Introduction.....	3
a.	L'objectif du projet.....	3
b.	Technologies et outils utilisés	3
c.	Résultats attendus	5
III.	Défis et solutions techniques	6
a.	Suivi de trajectoire autonome.....	6
b.	Évitement d'obstacles	6
c.	Tests et simulations	6
d.	Suivi de personnes et reconnaissance faciale	6
e.	Prototypage et améliorations	7
f.	Solutions techniques mises en place au final	7
	Outil de Mapping en Python	7
	Détection avec YOLO.....	8
g.	Aspect du projet abandonné	9
	La détection et le suivi de personne	9
	Test et simulations.....	9
	Suivi de trajectoire et évitement d'obstacle	9
IV.	Annexes.....	11
a.	Mapping et suivi de trajectoire code commenté	11
b.	Détection avec YoLo et suivi de personne	14

I. Introduction

a. L'objectif du projet

Problématiques :

L'objectif de ce projet est de programmer le drone Tello pour qu'il suive une trajectoire prédéfinie de manière autonome tout en évitant les obstacles en temps réel. Ce projet exploitera les capteurs du drone et utilisera des techniques de vision par ordinateur pour identifier et contourner des objets rencontrés sur son chemin.

Contexte : Dans un premier temps, ce cahier des charges expose les besoins et spécifications initiales du projet. Ensuite, il présente les décisions techniques retenues ainsi que leur mise en œuvre.

Objectif :

- Programmer une trajectoire pour le drone Tello (par exemple, un carré ou une trajectoire en zigzag).
- Utiliser la caméra du drone pour détecter et éviter des obstacles dynamiques ou statiques pendant le vol.
- Intégrer un système de contrôle autonome pour ajuster la trajectoire en fonction des Informations des capteurs

b. Technologies et outils utilisés

- Drone Tello pour le vol autonome.

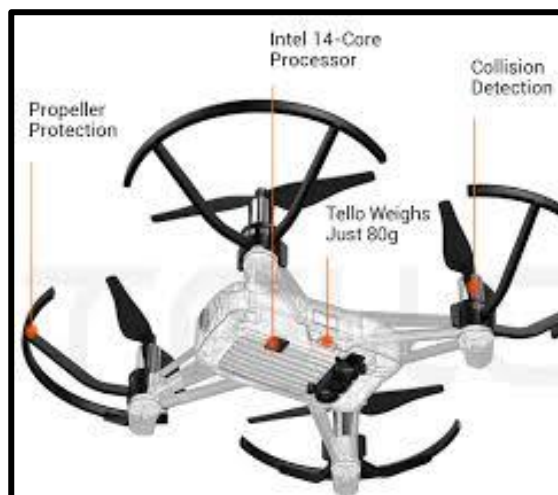


Figure 1 « Un schéma représentant un drone DJI Tello »

- b. Python avec la bibliothèque DJITelloPy pour contrôler le drone.



Figure 2 « Librairie DJI Tello pour python »

- c. Open CV pour la vision par ordinateur et la détection des obstacles.



Figure 3 « Librairie OpenCV »

- d. NumPy et autres bibliothèques Python pour le traitement des données.



Figure 4 « Librairie NumPy »

- e. Yolo pour classifier et détecter les éléments sur le retour vidéo.



Figure 5 « Librairie YoLO »

c. Résultats attendus

- a. Un système autonome pour le drone Tello qui lui permet de suivre une trajectoire prédéfinie tout en évitant les obstacles en temps réel.
- b. La capacité de détecter et contourner des obstacles dans un environnement dynamique sans intervention humaine.
- c. Un rapport détaillant les résultats obtenus, les défis rencontrés et les solutions apportées.

II. Défis et solutions techniques

a. Suivi de trajectoire autonome

Problématique : Absence de GPS intégré.

Solution envisagée :

- Utilisation de points visuels fixes définis dans l'espace.
- Le drone démarre depuis une position initiale (point 0) et estime sa position à l'aide des données de vitesse fournies par son capteur inertiel (IMU).

b. Évitement d'obstacles

Problématique : Absence de capteurs de distance sur le drone.

Solution envisagée :

- Utilisation d'un algorithme de détection d'obstacles basé sur l'analyse d'image.
- Intégration d'OpenCV pour la reconnaissance d'objets et l'évitement d'obstacles.
- Développement d'un algorithme de prise de décision en temps réel pour contourner les obstacles.

c. Tests et simulations

Problématique : Valider le fonctionnement du programme avant les essais en conditions réelles.

Solution envisagée :

- Installation d'un simulateur open-source pour tester le code en environnement virtuel.
- Exploration des solutions de simulation existantes compatibles avec le Tello.
- Si aucune solution de simulation n'est satisfaisante, tests réels avec des protocoles de sécurité adaptés.

d. Suivi de personnes et reconnaissance faciale

Problématique : Comment permettre au drone de suivre une personne spécifique pendant la phase de vol du drone.

Solution envisagée :

- Expérimentation avec OpenCV pour la détection faciale.
- Validation des performances de reconnaissance en conditions réelles.
- Ajustements des paramètres de détection pour éviter les faux positifs.

e. Prototypage et améliorations

Problématique : Permettre d'imprimer des protections autour des hélices lorsque l'on a une cassure pendant les essais.

- Impression 3D de protections d'hélices pour permettre des tests en toute sécurité.
- Optimisation du poids et de l'équilibrage du drone si nécessaire.

f. Solutions techniques mises en place au final

Outil de Mapping en Python

Un outil de mapping a été développé en Python pour permettre de tracer des points que le drone va suivre. Cet outil utilise la bibliothèque tkinter pour créer une interface graphique où l'utilisateur peut sélectionner des points sur une carte. Les points sont sauvegardés dans un fichier JSON pour être utilisés par le drone lors de son vol autonome.

Interface Graphique : L'outil affiche une carte (map.png) et permet de cliquer pour ajouter des points. Chaque point est visualisé avec un cercle et des flèches indiquant la trajectoire.

Calcul des Distances : Les distances entre les points sont calculées et affichées en mètres, facilitant la planification de la trajectoire.

Sauvegarde des Points : Les points peuvent être sauvegardés dans un fichier JSON (points.json) pour une utilisation ultérieure par le drone.

Fonctionnalités Supplémentaires : L'outil permet de générer un chemin aléatoire et de supprimer le dernier point ajouté, offrant ainsi une flexibilité dans la définition de la trajectoire.

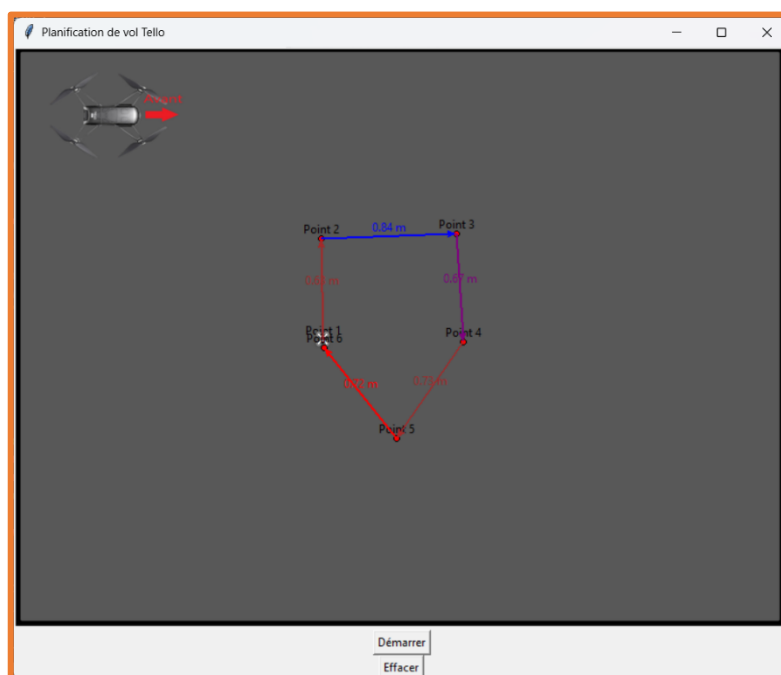


Figure 6 « Fenêtre qui permet de planifier une trajectoire au drone Tello et de le faire décoller directement depuis l'application »

Détection avec YOLO

Le système utilise YOLOv5 pour la détection des visages et des obstacles, permettant au drone de suivre une personne spécifique sans changer de cible et d'éviter les obstacles en temps réel. Cette fonctionnalité est essentielle pour les applications de suivi de personnes et de reconnaissance faciale.

- **Suivi de Personne : YOLOv5** permet d'identifier un individu et de suivre sa position en continu. C'est idéal pour des applications telles que la **surveillance** ou l'**assistance autonome**. Lors de nos tests, le drone a parfaitement suivi la personne dès qu'il l'a fixée comme cible. Cette fonctionnalité a bien fonctionné et a montré des résultats satisfaisants pour le suivi dynamique d'une personne, même avec des déplacements variés.

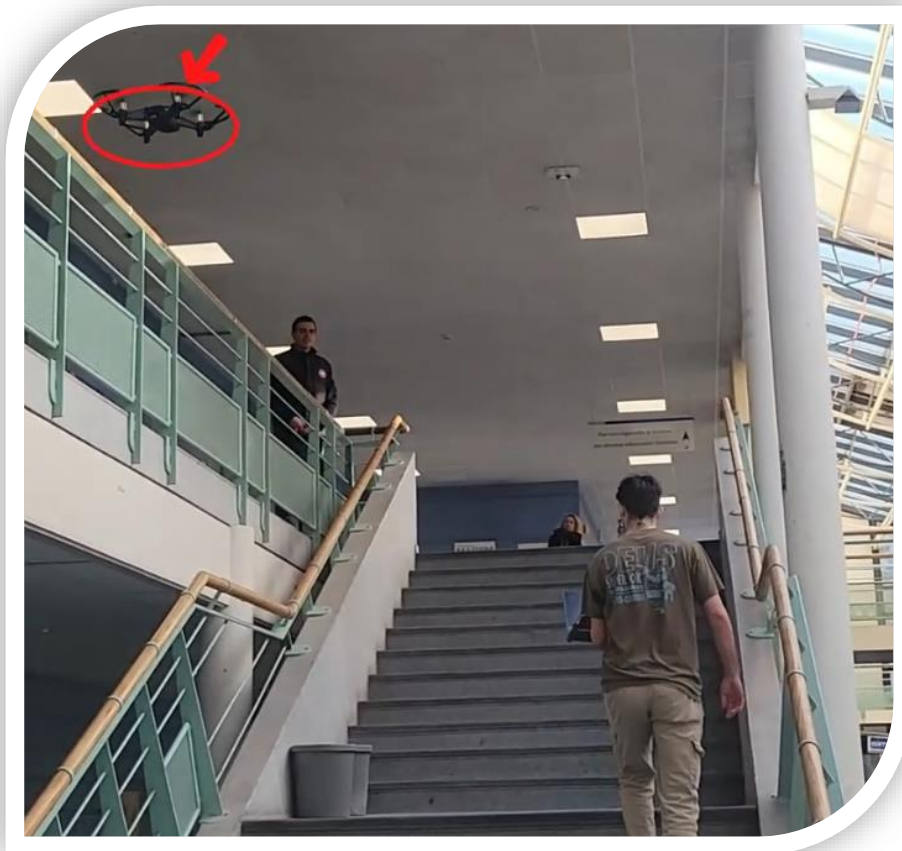


Figure 7 Suivi d'une personne par le DJI Tello dans les escaliers

g. Aspect du projet abandonné

Durant le projet, on a voulu ajouter différentes parties cependant durant la phase de R&D on s'est rendu compte de l'impossibilité ou d'une trop grosse difficulté pour mettre en place ces solutions.

La détection et le suivi de personne avec YOLO

Initialement, notre objectif était de permettre au drone de détecter et de suivre une personne de manière autonome en utilisant YOLO. Bien que YOLO permette de détecter plusieurs types d'objets, nous avons rapidement constaté que ce modèle ne permet pas de détecter des visages spécifiques. Pour résoudre ce problème, nous avons décidé d'entraîner un modèle personnalisé, en utilisant des images de nos visages, à travers une application conseillée par notre professeur. Le modèle s'est montré très performant lorsqu'il était testé avec une webcam d'ordinateur et dans les mêmes conditions que celles d'entraînement. Cependant, dès que les conditions de test changeaient (par exemple, avec des angles ou des éclairages différents), le modèle perdait sa précision. Ce problème nous a empêchés d'utiliser cette méthode pour une détection fiable en situation réelle, notamment en extérieur.

Test et simulations

Un autre aspect clé de notre projet était la simulation des tests avant de déployer le système sur le drone réel. Nous avons cherché à utiliser des simulateurs gratuits et open source en ligne pour tester nos algorithmes de détection et de suivi sans risquer d'endommager le drone ou d'avoir des problèmes pendant les essais. Cependant, nous n'avons pas réussi à trouver de simulateurs suffisamment complets et adaptés à nos besoins. En conséquence, nous avons dû procéder à des tests en direct, ce qui a augmenté les risques et les coûts associés au projet. Ce manque de simulateur fiable nous a également conduit à envisager la **modélisation de protections pour le drone** afin de sécuriser les tests en extérieur, mais cela n'a pas pu être mis en place dans le délai imparti.

Suivi de trajectoire et évitement d'obstacle

L'intégration de la détection d'obstacles en temps réel était essentielle pour assurer la sécurité du drone et maintenir une trajectoire stable pendant le suivi de personne. Toutefois, après plusieurs tentatives de mise en œuvre, nous avons constaté que OpenCV, bien qu'efficace pour détecter des objets, nécessite l'application de multiples algorithmes de détection pour que le drone puisse éviter les obstacles de manière fluide et précise. Ces algorithmes doivent être suffisamment sophistiqués pour gérer les situations en temps réel, ce qui nécessite des ajustements continus du comportement du drone en fonction de son environnement. Nous n'avons malheureusement pas eu le temps nécessaire pour développer et tester ces algorithmes dans les meilleures conditions, ce qui a conduit à l'abandon de cette fonctionnalité. Malgré tout, l'idée était de créer un système d'évitement en temps réel, mais les contraintes techniques et temporelles ont eu raison de ce projet.

III. Conclusion

Ce projet visait à programmer un drone Tello pour qu'il suive une trajectoire prédéfinie tout en évitant les obstacles. En utilisant Python, DJITelloPy, OpenCV et YOLO, nous avons développé un système capable de guider le drone de manière autonome. Les principaux défis, tels que l'absence de GPS et de capteurs de distance, ont été surmontés grâce à des solutions innovantes. Les tests réalisés ont permis de valider le fonctionnement du système, bien que certaines limitations aient été identifiées. Les résultats obtenus montrent le potentiel des drones autonomes dans diverses applications. Ce projet a également mis en lumière l'importance de la recherche continue pour améliorer les performances des systèmes autonomes. En conclusion, ce projet a atteint ses objectifs et ouvre la voie à de futures améliorations dans le domaine des drones autonomes.

IV. Annexes

a. Mapping et suivi de trajectoire code commenté

```
# -*- coding: utf-8 -*-
"""
Script permettant de planifier et suivre la trajectoire d'un drone DJI
Tello en utilisant une interface graphique Tkinter.
Le drone suit les points cliqués sur une carte affichée dans l'interface.

"""

import tkinter as tk
from PIL import Image, ImageTk
from djitellopy import Tello
import math
import time
import random

# Initialisation du drone
# Connexion au drone Tello et affichage du niveau de batterie
tello = Tello()
tello.connect()
print(f"Batterie : {tello.get_battery()}%")

# Paramètres généraux pour la conversion et le déplacement
PIXEL_TO_CM = 0.6 # Conversion des pixels en centimètres
MAX_MOVE_CM = 500 # Distance maximale que le drone peut parcourir en une
seule commande
arrow_colors = ["blue", "green", "red", "purple", "orange", "brown"] #
Couleurs pour les flèches

# Stockage des points de trajectoire
points = [] # Liste des points cliqués
point_visuals = [] # Liste des objets graphiques associés aux points
origin = None # Point d'origine (premier point cliqué)

# Fonction pour calculer l'angle entre la position actuelle et la cible
def calculate_angle_to_target(current_x, current_y, target_x, target_y):
    dx = target_x - current_x
    dy = target_y - current_y
    return math.degrees(math.atan2(dy, dx))

# Fonction pour normaliser un angle entre -180 et 180 degrés
def normalize_angle(angle):
    while angle > 180:
        angle -= 360
    while angle < -180:
        angle += 360
    return angle

# Fonction de déplacement du drone en suivant la trajectoire définie
def move_drone():
    tello.takeoff() # Décollage du drone
    time.sleep(1)
    prev_x, prev_y, pre_angle = 0, 0, 0 # Initialisation de la position et
de l'angle

    for i in range(1, len(points)):
        target_x, target_y = points[i]
```

```

        target_x_cm, target_y_cm = target_x * PIXEL_TO_CM, target_y *
PIXEL_TO_CM

        # Calcul de l'angle nécessaire pour atteindre le prochain point
        target_angle = calculate_angle_to_target(prev_x, prev_y,
target_x_cm, target_y_cm)
        rotation_angle = normalize_angle(target_angle - pre_angle)

        # Rotation du drone dans la direction de la cible
        if rotation_angle > 0:
            tello.rotate_clockwise(int(rotation_angle))
        else:
            tello.rotate_counter_clockwise(int(abs(rotation_angle)))

        pre_angle = target_angle

        # Calcul de la distance à parcourir
        distance = math.sqrt((target_x_cm - prev_x)**2 + (target_y_cm -
prev_y)**2)

        # Si la distance dépasse la limite, diviser le trajet en plusieurs
étapes
        while distance > MAX_MOVE_CM:
            tello.move_forward(MAX_MOVE_CM)
            distance -= MAX_MOVE_CM
            canvas.coords(drone_visual, origin[0] + (prev_x + MAX_MOVE_CM /
PIXEL_TO_CM), origin[1] + prev_y)
            root.update()
            time.sleep(1)

            tello.move_forward(int(distance)) # Déplacement final vers le
point cible
            time.sleep(1)
            canvas.coords(drone_visual, origin[0] + target_x, origin[1] +
target_y)
            root.update()

            prev_x, prev_y = target_x_cm, target_y_cm # Mise à jour de la
position actuelle

        tello.land() # Atterrissage du drone

# Fonction pour tracer une flèche indiquant la distance entre deux points
def draw_arrow_with_distance(start, end, color):
    line = canvas.create_line(*start, *end, arrow=tk.LAST, fill=color,
width=2)
    distance = math.sqrt((end[0] - start[0])**2 + (end[1] - start[1])**2) *
PIXEL_TO_CM / 100
    distance_text = canvas.create_text((start[0] + end[0]) / 2, (start[1] +
end[1]) / 2 - 10, text=f"{round(distance,2)} m", fill=color)
    return line, distance_text

# Fonction pour ajouter un point sur la carte
def add_point(x, y):
    global origin, drone_visual
    if origin is None:
        origin = (x, y)
    adjusted_x = x - origin[0]
    adjusted_y = y - origin[1]
    points.append((adjusted_x, adjusted_y))

```

```

    # Création des représentations graphiques des points
    point_visuals.append(canvas.create_oval(x-3, y-3, x+3, y+3,
fill='red'))
    text_visual = canvas.create_text(x, y-10, text=f"Point {len(points)}",
fill="black")
    point_visuals.append(text_visual)

    # Affichage de l'image du drone au premier point
    if len(points) == 1:
        drone_visual = canvas.create_image(x, y, anchor=tk.CENTER,
image=drone_photo)
        point_visuals.append(drone_visual)

    # Tracer une flèche entre les points successifs
    if len(points) > 1:
        start = (origin[0] + points[-2][0], origin[1] + points[-2][1])
        end = (origin[0] + points[-1][0], origin[1] + points[-1][1])
        color = random.choice(arrow_colors)
        arrow, distance_text = draw_arrow_with_distance(start, end, color)
        point_visuals.extend([arrow, distance_text])

# Fonction appelée lors du clic sur la carte
def click_event(event):
    add_point(event.x, event.y)

# Fonction pour réinitialiser les points et la trajectoire
def clear_points():
    global origin, drone_visual
    points.clear()
    origin = None
    for visual in point_visuals:
        canvas.delete(visual)
    point_visuals.clear()
    if drone_visual:
        canvas.delete(drone_visual)
        drone_visual = None

# Création de l'interface Tkinter
root = tk.Tk()
root.title("Planification de vol Tello")

# Chargement et affichage de la carte
image = Image.open("MAP/map.png").resize((800, 600), Image.LANCZOS)
photo = ImageTk.PhotoImage(image)
canvas = tk.Canvas(root, width=800, height=600)
canvas.pack()
canvas.create_image(0, 0, anchor=tk.NW, image=photo)
canvas.bind("<Button-1>", click_event)

# Chargement de l'image du drone
drone_image = Image.open("MAP/drone.png").resize((70, 60), Image.LANCZOS)
drone_photo = ImageTk.PhotoImage(drone_image)

drone_visual = None # Initialisation de l'image du drone

# Boutons pour démarrer le vol et effacer la trajectoire
start_button = tk.Button(root, text="Démarrer", command=move_drone)
start_button.pack()
clear_button = tk.Button(root, text="Effacer", command=clear_points)
clear_button.pack()

```



```
root.mainloop()
```

b. Détection avec YoLo et suivi de personne

```
import cv2
import numpy as np
import torch
from djitellopy import Tello
import time
import warnings
import math
import keyboard # Détection des touches du clavier

# Ignorer les avertissements futurs
warnings.simplefilter("ignore", FutureWarning)

start_time = time.time()

# Paramètres d'affichage
screen_width = int(1920/4) # Définir la largeur de l'écran
screen_height = int(1080/4)
cv2.namedWindow("Output", cv2.WINDOW_NORMAL)
cv2.resizeWindow("Output", screen_width, screen_height)

# Chargement du modèle YOLOv5 pour la détection d'objets (ici des visages)
model = torch.hub.load('ultralytics/yolov5', 'yolov5n', pretrained=True)

# Initialisation du drone DJI Tello
tello1 = Tello()
tello1.connect()
print('Niveau de batterie:', tello1.get_battery())
tello1.streamon()
time.sleep(2)
tello1.takeoff()
time.sleep(3)

# Paramètres pour le suivi
w, h = 360, 240 # Taille de l'image de suivi
fbRange = [50, 100] # Plage de distances acceptables
pid = [0.4, 0.4, 0] # Paramètres du PID pour le contrôle
target = False # Cible initiale non détectée
pError = 0 # Erreur précédente pour le PID

# Fonction de détection des visages avec YOLOv5
def findFace(img, target):
    results = model(img) # Détection avec YOLO
    detections = results.xyxy[0].numpy() # Récupération des détections
    sous forme de tableau
    clear_detections = detections[detections[:, 5] == 0] # Filtrage pour
    ne garder que les visages

    best_conf = 0 # Meilleure confiance détectée

    if target == False: # Si aucune cible n'est suivie, en choisir une
        for clear_detection in clear_detections:
            x1, y1, x2, y2, conf, cls = clear_detection
            if conf > 0.5 and cls == 0 and best_conf < conf:
                best_conf = conf
```

```

        target = [x1, y1, x2, y2, conf, cls]

    if target != False: # Suivi d'une cible déjà sélectionnée
        dist_min = np.inf
        new_target = target
        for i in range(len(clear_detections)):
            if clear_detections[i][4] > 0.4 and clear_detections[i][5] ==
0:
                x1, y1, x2, y2, conf, cls = clear_detections[i]
                distance = math.dist((target[0], target[1]), (x1, y1)) +
math.dist((target[2], target[3]), (x2, y2))
                if dist_min > distance:
                    dist_min = distance
                    new_target = [x1, y1, x2, y2, conf, cls]
        target = new_target

    x1, y1, x2, y2, conf, cls = target
    if conf > 0.5 and cls == 0:
        cx, cy = (x1 + x2) // 2, (y1 + y2) // 2 # Centre du visage
        area = (x2 - x1) * (y2 - y1) # Aire du visage détecté
        bestFace = (cx, cy, area, x1, y1, x2, y2)
        cv2.rectangle(img, (x1, y1), (x2, y2), (0, 255, 0), 2)
        cv2.circle(img, (cx, cy), 5, (0, 0, 255), cv2.FILLED)
        return img, [[cx, cy], x2-x1, y1], target, "Personne detectee"

    return img, [[0, 0], 0, 45], target, "Aucune Personne detectee"

# Fonction pour ajuster la position du drone en fonction de la détection
def trackFace(info, w, pid, pError):
    area = info[1]
    x, y = info[0]
    fb = 0 # Commande de déplacement avant/arrière
    error = x - w // 2
    yaw = pid[0] * error + pid[1] * (error - pError) if pError else 0
    yaw = int(np.clip(yaw, -100, 100))

    y_haut = info[-1]
    if fbRange[0] < area < fbRange[1]:
        fb = 0
    elif area > fbRange[1]:
        fb = -20 # Reculer
    elif area < fbRange[0]:
        fb = 35 # Avancer

    fly_up = 50 if y_haut <= 40 else (-50 if y_haut >= 100 else 0)

    tello1.send_rc_control(0, fb, fly_up, yaw)
    return error

# Fonction d'affichage du statut et des infos du drone
def hud(img, detection_status):
    battery_level = tello1.get_battery()
    elapsed_time = int(time.time() - start_time) # Temps écoulé en
secondes
    cv2.putText(img, f"Temps de vol: {elapsed_time}s", (250, 10),
cv2.FONT_HERSHEY_SIMPLEX, 0.35, (255, 0, 0), 1)
    cv2.putText(img, f"{battery_level}%", (5, 10),
cv2.FONT_HERSHEY_SIMPLEX, 0.35, (51, 204, 51), 1)
    color = (0, 255, 0) if detection_status == "Personne detectee" else (0,
0, 255)

```

```

        cv2.putText(img, detection_status, (10, h - 20),
cv2.FONT_HERSHEY_SIMPLEX, 0.35, color, 1)
        return img

# Boucle principale pour la capture et le traitement vidéo
while True:
    img = tello1.get_frame_read().frame # Capture du flux vidéo
    if img is None:
        print("\n⚠️ Aucun cadre reçu, tentative de reconnexion...")
        time.sleep(0.1)
        continue
    img = cv2.resize(img, (w, h))
    img, info, target, detection_status = findFace(img, target)
    img = hud(img, detection_status)
    pError = trackFace(info, w, pid, pError)
    cv2.imshow("Output", img)

    if keyboard.is_pressed('esc'): # Quitter avec Échap
        break

# Arrêt du drone et fermeture des fenêtres
if tello1.is_flying:
    tello1.land()
tello1.streamoff()
cv2.destroyAllWindows()
print(" Programme terminé.")

```